

CS232/CS332 Introduction to Cloud Computing Technology

Module 06: Compute (Part 2)

**By Asst.Prof.Dr. Prapaporn Rattanatamrong
and Asst.Prof.Dr. Wanida Putthividhya**

Department of Computer Science

Faculty of Science & Technology, Thammasat University

Last update: January 2026

Lesson Learning Outcomes

After this module ends, students must be able to:

- Describe various AWS compute services, including Amazon EC2, AWS Lambda, AWS Elastic Beanstalk, and container services.
- Demonstrate the ability to launch, configure, and manage Amazon EC2 instances using the AWS Management Console.
- Identify cost optimization strategies for Amazon EC2, including choosing the appropriate pricing model (On-Demand, Reserved, Spot, etc.) and right-sizing instances.
- Explain the benefits of AWS Lambda, create and deploy Lambda functions, and integrate them with AWS services.
- Demonstrate how to deploy and manage web applications using AWS Elastic Beanstalk, understanding its automation capabilities for scaling and deployment.

Section 3: Amazon EC2 cost optimization

Module 6: Compute

Amazon EC2 pricing models

On-Demand Instances

- Pay by the hour
- No long-term commitments.
- Eligible for the [AWS Free Tier](#).

Dedicated Hosts

- A physical server with EC2 instance capacity fully dedicated to your use.

Dedicated Instances

- Instances that run in a VPC on hardware that is dedicated to a single customer.

Reserved Instances

- Full, partial, or no upfront payment for instance you reserve.
- Discount on hourly charge for that instance.
- 1-year or 3-year term.

Scheduled Reserved Instances

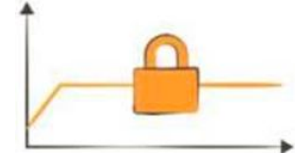
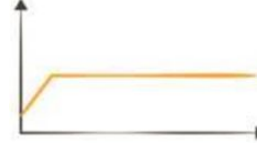
- Purchase a capacity reservation that is always available on a recurring schedule you specify.
- 1-year term.

Spot Instances

- Instances run as long as they are available and your bid is above the Spot Instance price.
- They can be interrupted by AWS with a 2-minute notification.
- Interruption options include terminated, stopped or hibernated.
- Prices can be significantly less expensive compared to On-Demand Instances
- Good choice when you have flexibility in when your applications can run.

Per second billing available for On-Demand Instances, Reserved Instances, and Spot Instances that run Amazon Linux or Ubuntu.

Amazon EC2 pricing models: Benefits



On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
• Low cost and flexibility	• Large scale, dynamic workload	• Predictability ensures compute capacity is available when needed	• Save money on licensing costs • Help meet compliance and regulatory requirements

Amazon EC2 pricing models: Use cases



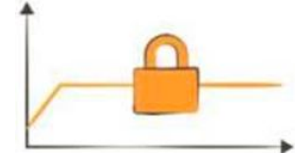
Spiky Workloads



Time-Insensitive Workloads



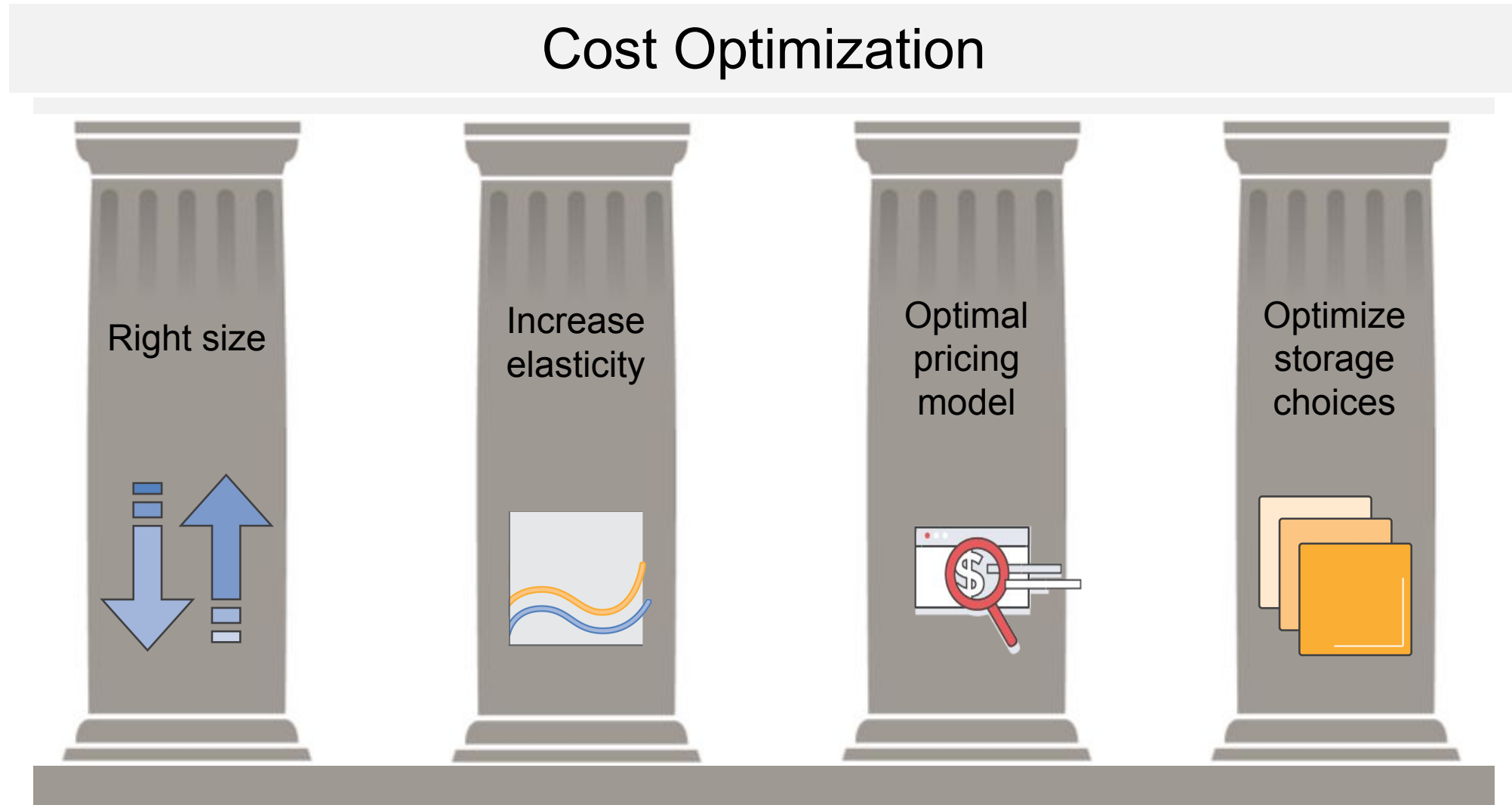
Steady-State Workloads



Highly Sensitive Workloads

On-Demand Instances	Spot Instances	Reserved Instances	Dedicated Hosts
• Short-term, spiky, or unpredictable workloads • Application development or testing	• Applications with flexible start and end times • Applications only feasible at very low compute prices • Users with urgent computing needs for large amounts of additional capacity	• Steady state or predictable usage workloads • Applications that require reserved capacity, including disaster recovery • Users able to make upfront payments to reduce total computing costs even further	• Bring your own license (BYOL) • Compliance and regulatory restrictions • Usage and licensing tracking • Control instance placement

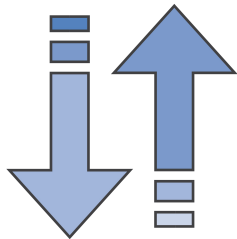
The four pillars of cost optimization



Pillar 1: Right size

Pillars

- :
1. Right size
 2. Increase elasticity
 3. Optimal pricing model
 4. Optimize storage choices



✓ Provision instances to match the need

- CPU, memory, storage, and network throughput
- Select appropriate **instance types** for your use

✓ Use Amazon CloudWatch metrics

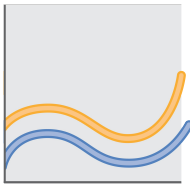
- How idle are instances? When?
- Downsize instances

✓ Best practice: Right size, then reserve

Pillar 2: Increase elasticity

Pillars

- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices



- ✓ **Stop** or **hibernate** Amazon EBS-backed instances that are not actively in use
 - Example: non-production development or test instances
- ✓ Use **automatic scaling** to match needs based on usage
 - Automated and time-based elasticity

Pillar 3: Optimal pricing model

Pillars

- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices

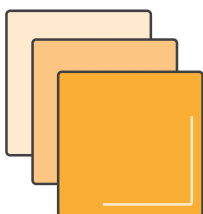


- ✓ Leverage the right pricing model for your use case
 - Consider your usage patterns
- ✓ Optimize and *combine* purchase types
- ✓ Examples:
 - Use **On-Demand Instance** and **Spot Instances** for variable workloads
 - Use **Reserved Instances** for predictable workloads
- ✓ Consider serverless solutions (AWS Lambda)

Pillar 4: Optimize storage choices

Pillars

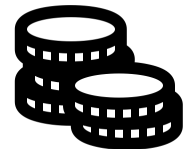
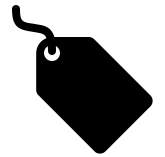
- 1. Right-Size
- 2. Increase Elasticity
- 3. Optimal pricing model
- 4. Optimize storage choices



- ✓ Reduce costs while maintaining storage performance and availability
- ✓ Resize EBS volumes
- ✓ Change EBS volume types
 - ✓ Can you meet performance requirements with less expensive storage?
 - ✓ Example: **Amazon EBS Throughput Optimized HDD (st1)** storage typically costs half as much as the default **General Purpose SSD (gp2)** storage option.
- ✓ Delete EBS snapshots that are no longer needed
- ✓ Identify the most appropriate destination for specific types of data
 - ✓ Does the application need the instance to reside on Amazon EBS?
 - ✓ Amazon S3 storage options with lifecycle policies can reduce costs

Measure, monitor, and improve

- Cost optimization is an ongoing process.
- Recommendations –
 - Define and enforce **cost allocation tagging**.
 - Define metrics, set targets, and review regularly.
 - Encourage teams to **architect for cost**.
 - Assign the responsibility of optimization to an individual or to a team.



Section 3 key takeaways



- **Amazon EC2 pricing models** include On-Demand Instances, Reserved Instances, Spot Instances, Dedicated Instances, and Dedicated Hosts.
- **Spot Instances** can be interrupted with a 2-minute notification. However, they can offer significant cost savings over On-Demand Instances.
- The **four pillars of cost optimization** are:
 - Right size
 - Increase elasticity
 - Optimal pricing model
 - Optimize storage choices

Activity 4:

"EC2 Pricing Models"

(15 minutes)

Scenario 1: What's the best pricing model?

Scenario: A startup needs to run a web application 24/7, but they are uncertain about their future traffic and resource requirements.



Scenario 2: What's the best pricing model?

Scenario: A research team needs to run a large-scale data analysis job periodically, which requires significant compute power but can tolerate interruptions.



Scenario 3: What's the best pricing model?

Scenario: A financial institution needs to run critical applications that require high availability, low latency, and compliance with regulatory standards.



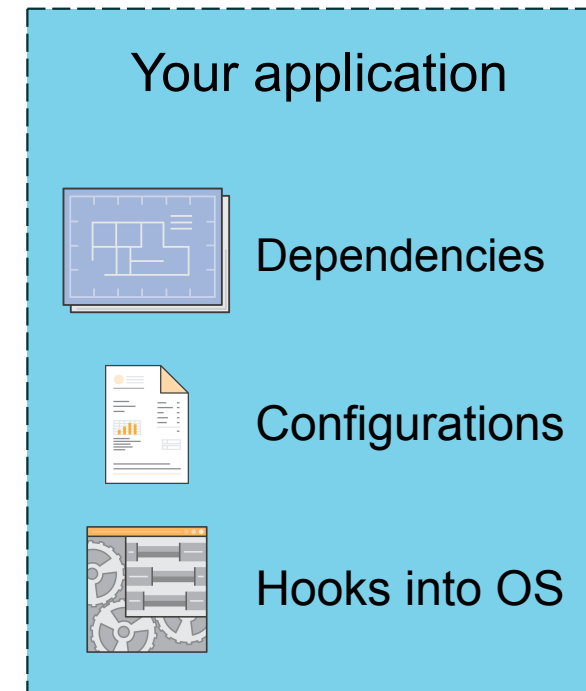
Section 4: Container services

Module 6: Compute

Container basics

- **Containers** are a method of operating system virtualization.
- **Benefits** –
 - Repeatable.
 - Self-contained environments.
 - Software runs the same in different environments.
 - Developer's laptop, test, production.
 - Faster to launch and stop or terminate than virtual machines

Your Container

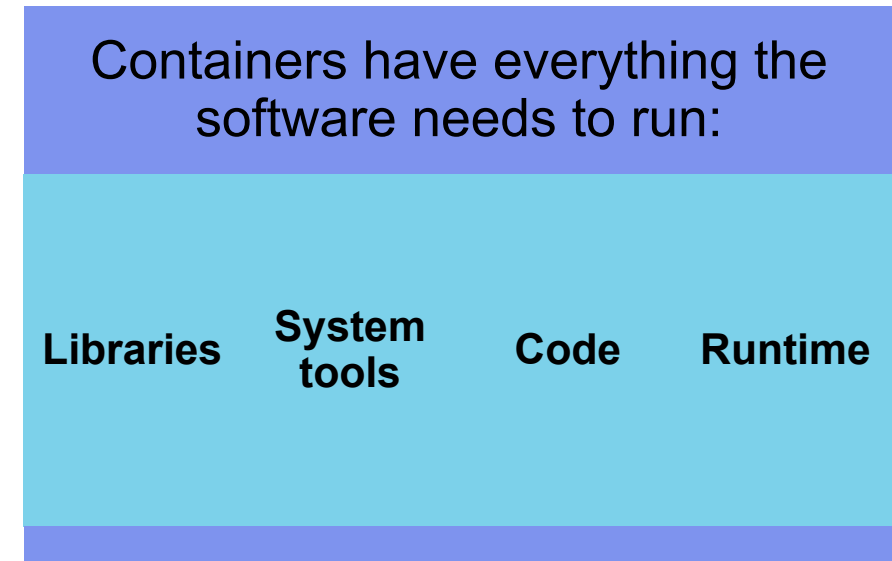


What is Docker?

- **Docker** is a software platform that enables you to build, test, and deploy applications quickly.
- You run containers on Docker.
 - Containers are created from a template called an *image*.
- A **container** has everything a software application needs to run.

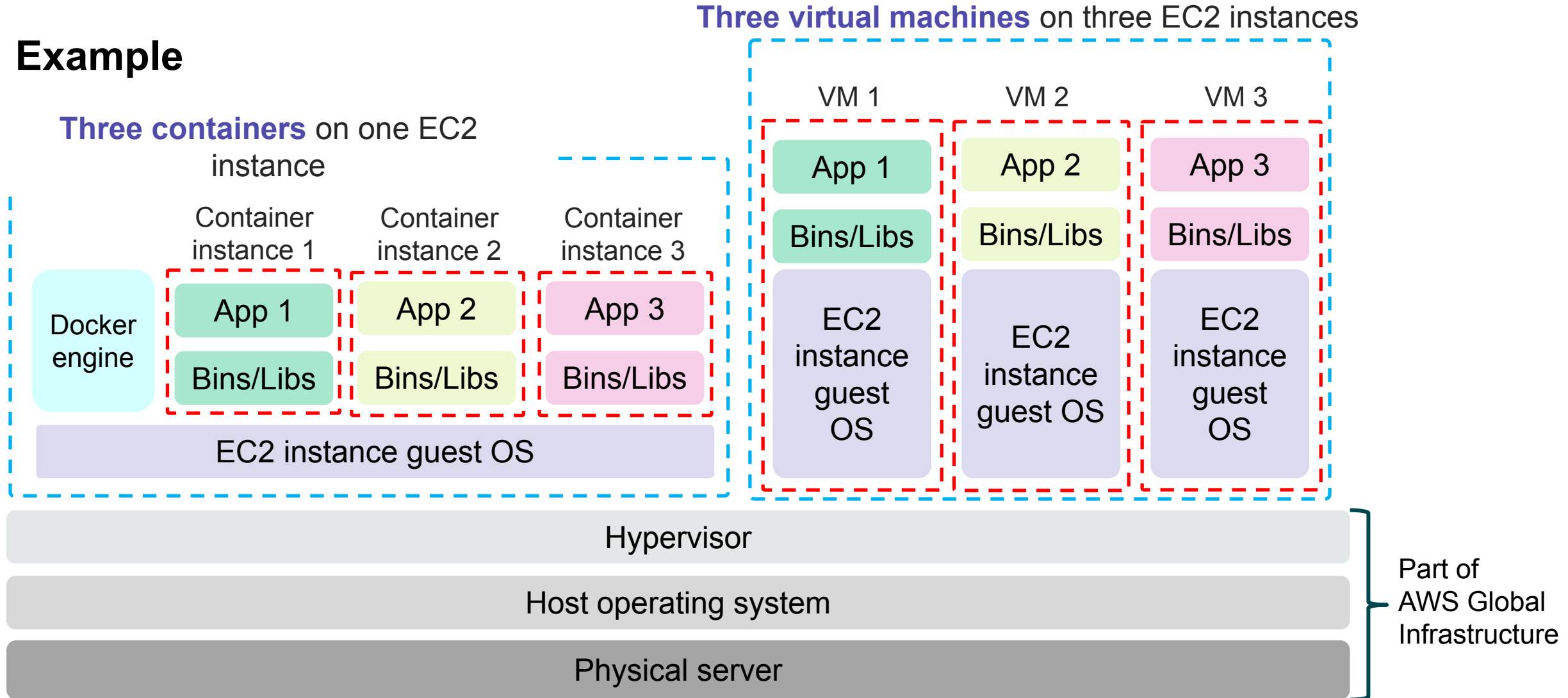


Container



Containers versus virtual machines

Example



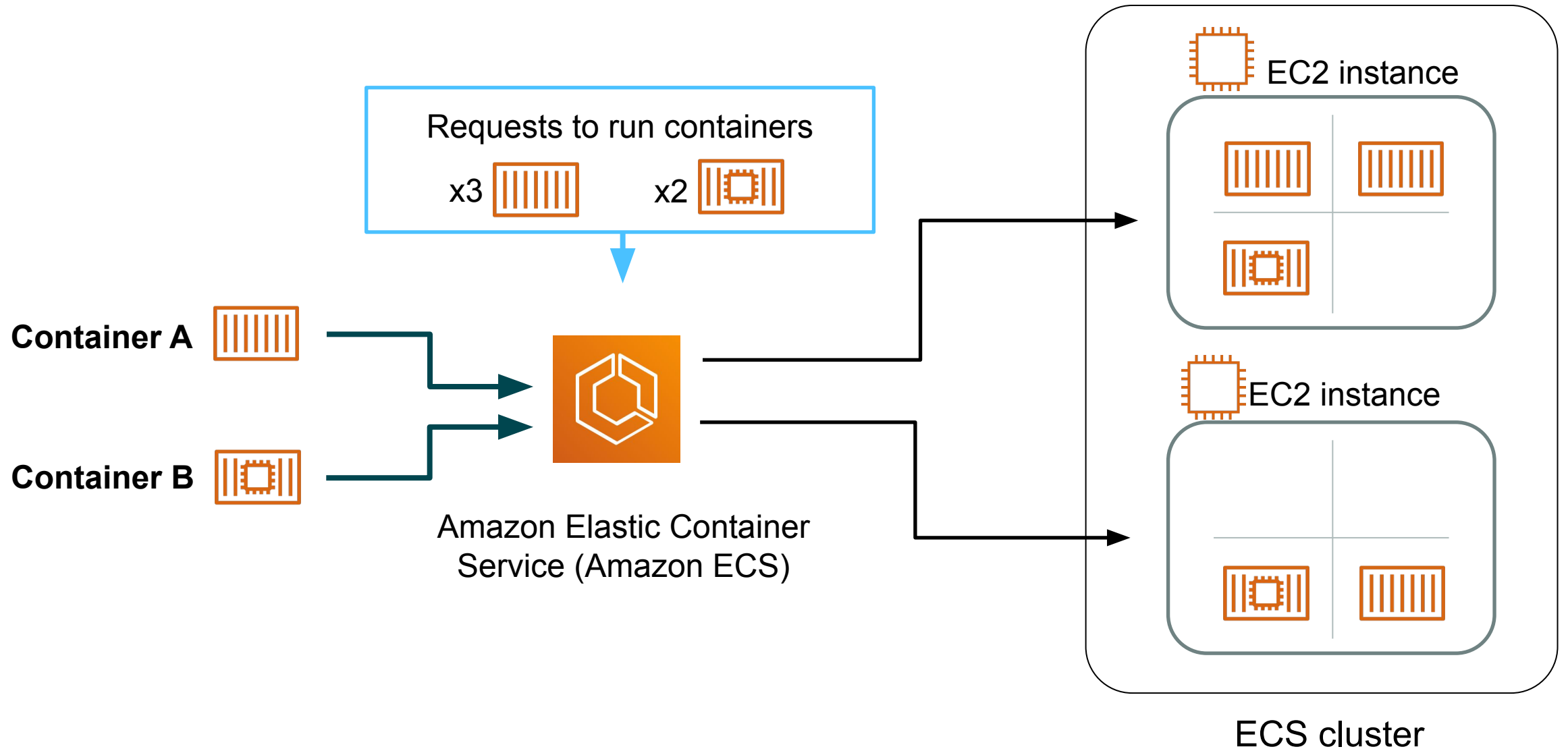
Amazon Elastic Container Service (Amazon ECS)

- Amazon Elastic Container Service (**Amazon ECS**) –
 - A highly scalable, fast, **container management service**
- Key benefits –
 - Orchestrates the running of Docker containers
 - Maintains and scales the fleet of nodes that run your containers
 - Removes the complexity of standing up the infrastructure
- Integrated with features that are familiar to Amazon EC2 service users –
 - Elastic Load Balancing
 - Amazon EC2 security groups
 - Amazon EBS volumes
 - IAM roles



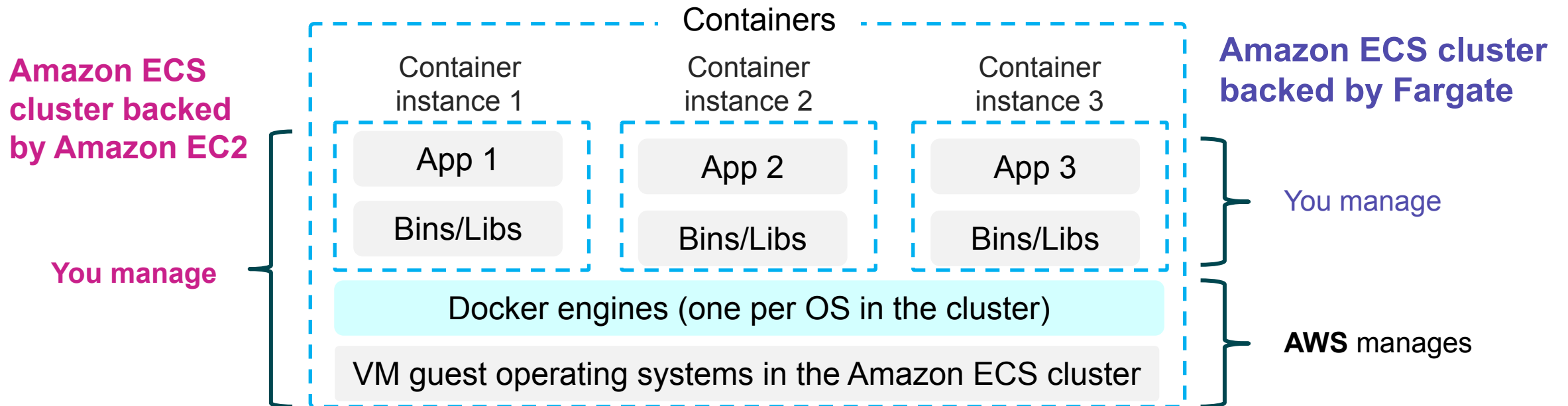
**Amazon Elastic
Container Service**

Amazon ECS orchestrates containers



Amazon ECS cluster options

- **Key question:** Do *you* want to manage the Amazon ECS cluster that runs the containers?
- If **yes**, create an **Amazon ECS cluster backed by Amazon EC2** (provides more granular control over infrastructure)
- If **no**, create an **Amazon ECS cluster backed by AWS Fargate** (easier to maintain, focus on your applications)



What is Kubernetes?

- Kubernetes is open source software for container orchestration.
 - Deploy and **manage containerized applications** *at scale*.
 - The same toolset can be used on premises and in the cloud.
- Complements Docker.
 - Docker enables you to run multiple containers on a single OS host.
 - Kubernetes **orchestrates** multiple Docker hosts (nodes).
- Automates –
 - Container provisioning.
 - Networking.
 - Load distribution.
 - Scaling.

Amazon Elastic Kubernetes Service (Amazon EKS)

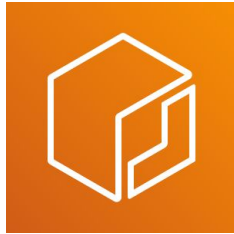
- Amazon Elastic Kubernetes Service (**Amazon EKS**)
 - Enables you to run Kubernetes on AWS
 - Certified Kubernetes conformant (supports easy migration)
 - Supports Linux and Windows containers
 - Compatible with Kubernetes community tools and supports popular Kubernetes add-ons
- Use Amazon EKS to –
 - Manage clusters of Amazon EC2 compute instances
 - Run containers that are orchestrated by Kubernetes on those instances



**Amazon Elastic
Kubernetes Service**

Amazon Elastic Container Registry (Amazon ECR)

Amazon ECR is a fully managed Docker **container registry** that makes it easy for developers to store, manage, and deploy Docker container images.



**Amazon Elastic
Container Registry**



Image



Registry

Amazon ECS integration

Docker support

Team collaboration

Access control

Third-party integrations

Section 4 key takeaways



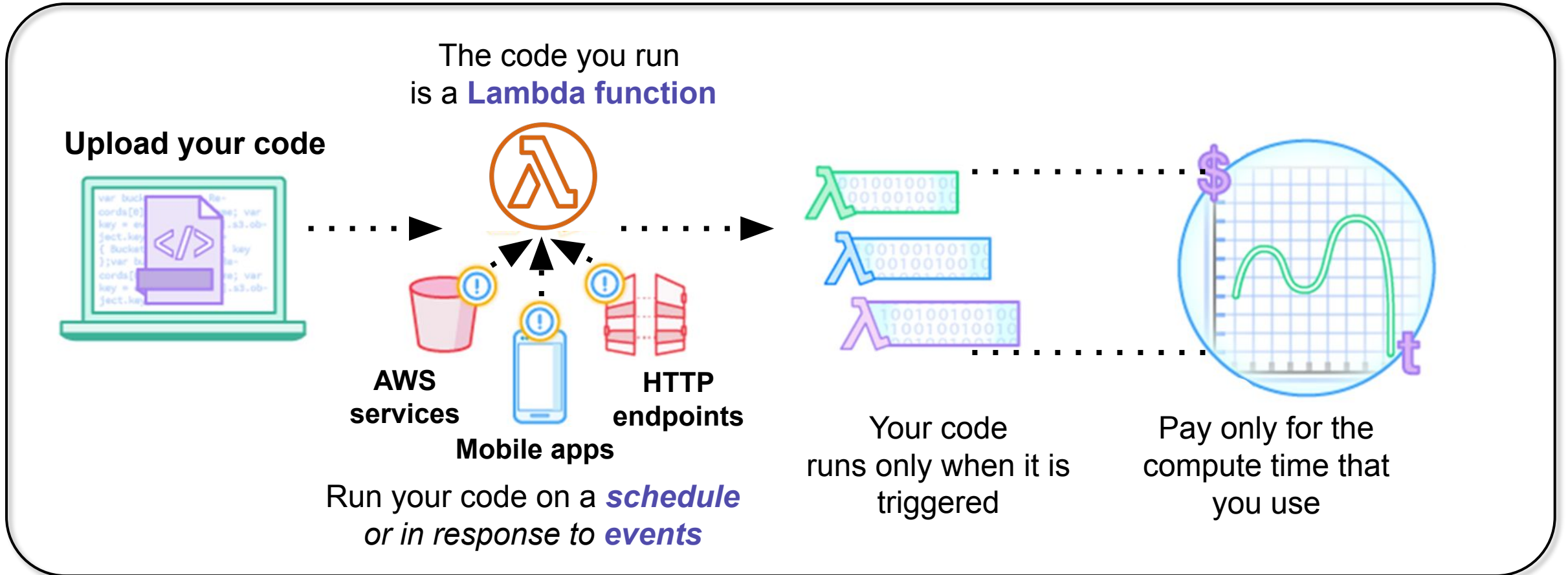
- **Containers** can hold everything that an application needs to run.
- **Docker** is a software platform that packages software into containers.
 - A single application can span multiple containers.
- Amazon Elastic Container Service (**Amazon ECS**) orchestrates the running of Docker containers.
- **Kubernetes** is open source software for container orchestration.
- Amazon Elastic Kubernetes Service (**Amazon EKS**) enables you to run Kubernetes on AWS
- Amazon Elastic Container Registry (**Amazon ECR**) enables you to store, manage, and deploy your Docker containers.

Section 5: Introduction to AWS Lambda

Module 6: Compute

AWS Lambda: Run code without servers

AWS Lambda is a **serverless** compute service.



Benefits of Lambda



**AWS
Lambda**



It supports multiple programming languages



Completely automated administration



Built-in fault tolerance



It supports the orchestration of multiple functions



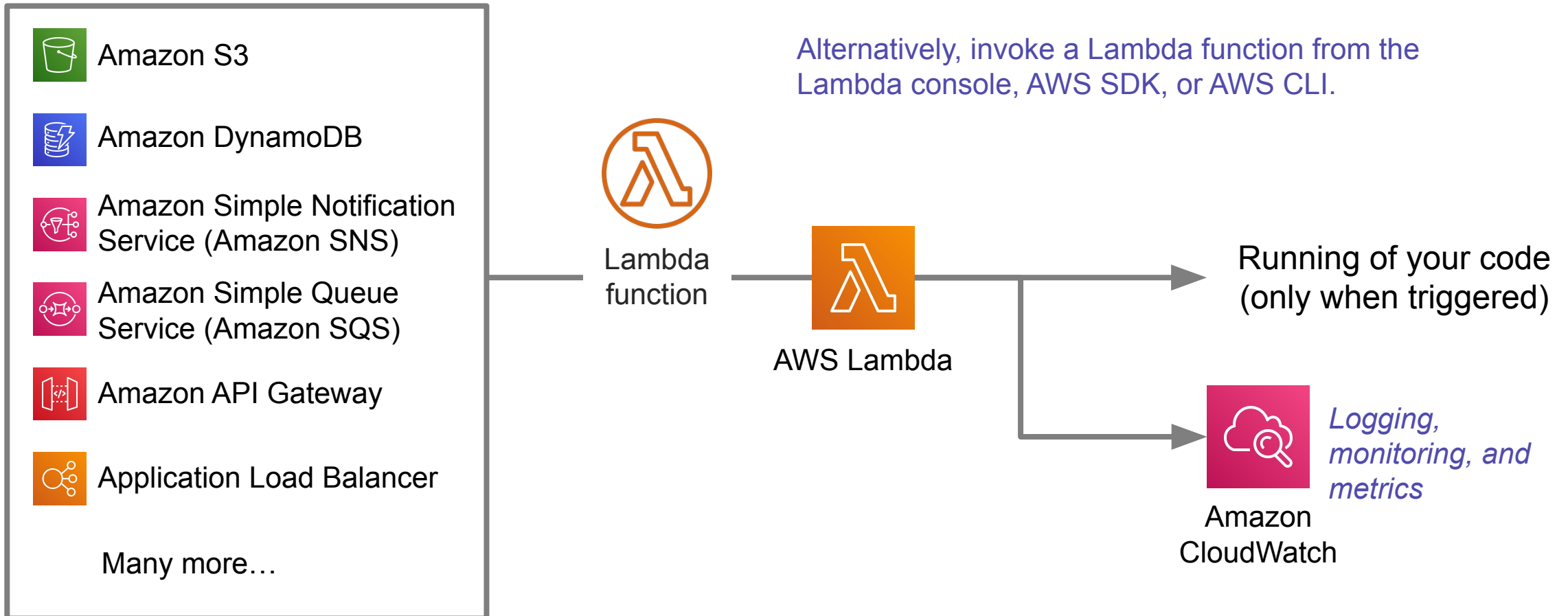
Pay-per-use pricing

AWS Lambda event sources

Event sources

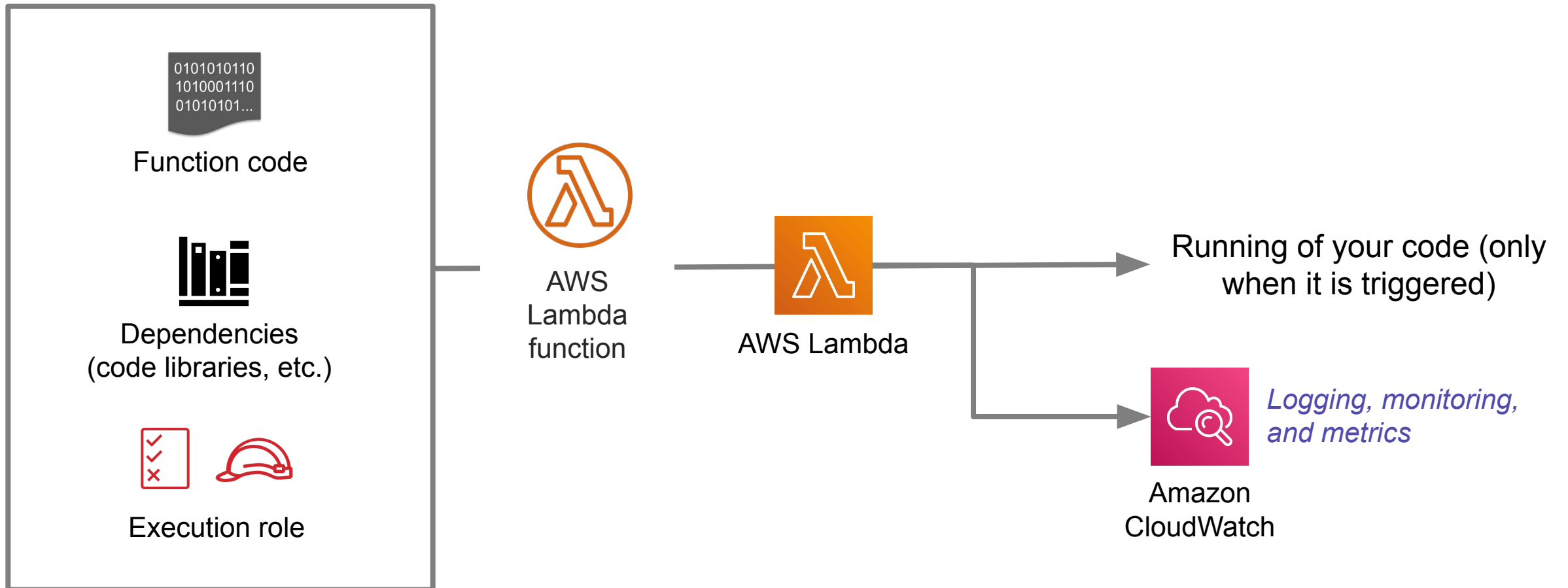
Configure other AWS services as **event sources** to invoke your function as shown here.

Alternatively, invoke a Lambda function from the Lambda console, AWS SDK, or AWS CLI.



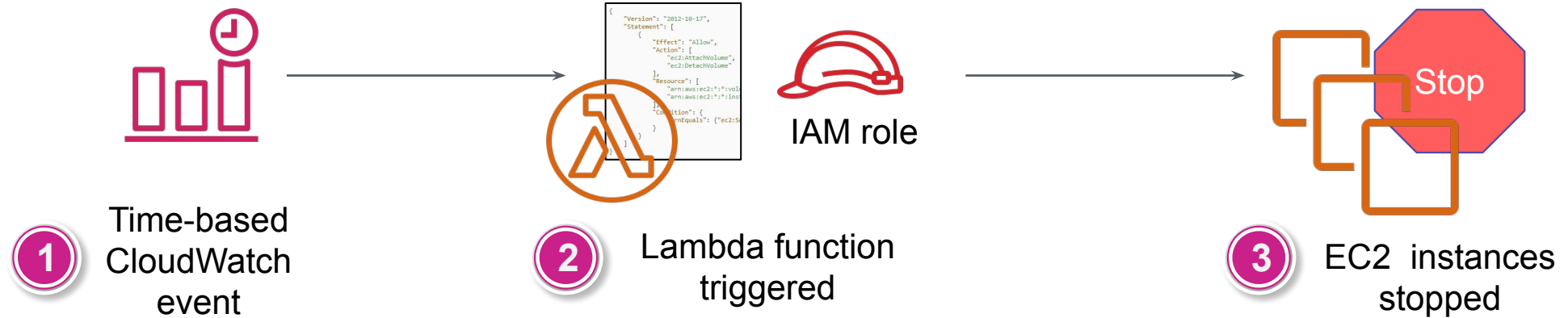
AWS Lambda function configuration

Lambda function configuration

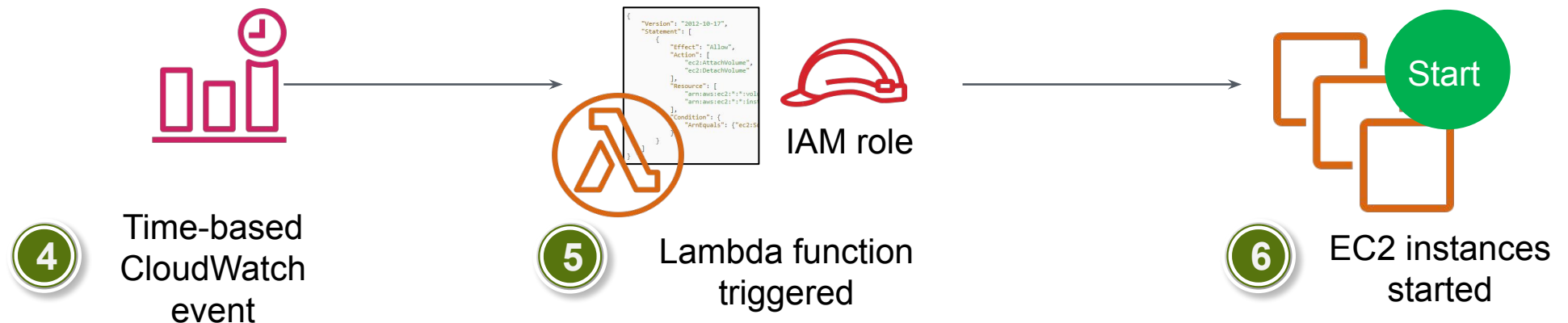


Schedule-based Lambda function example: Start and stop EC2 instances

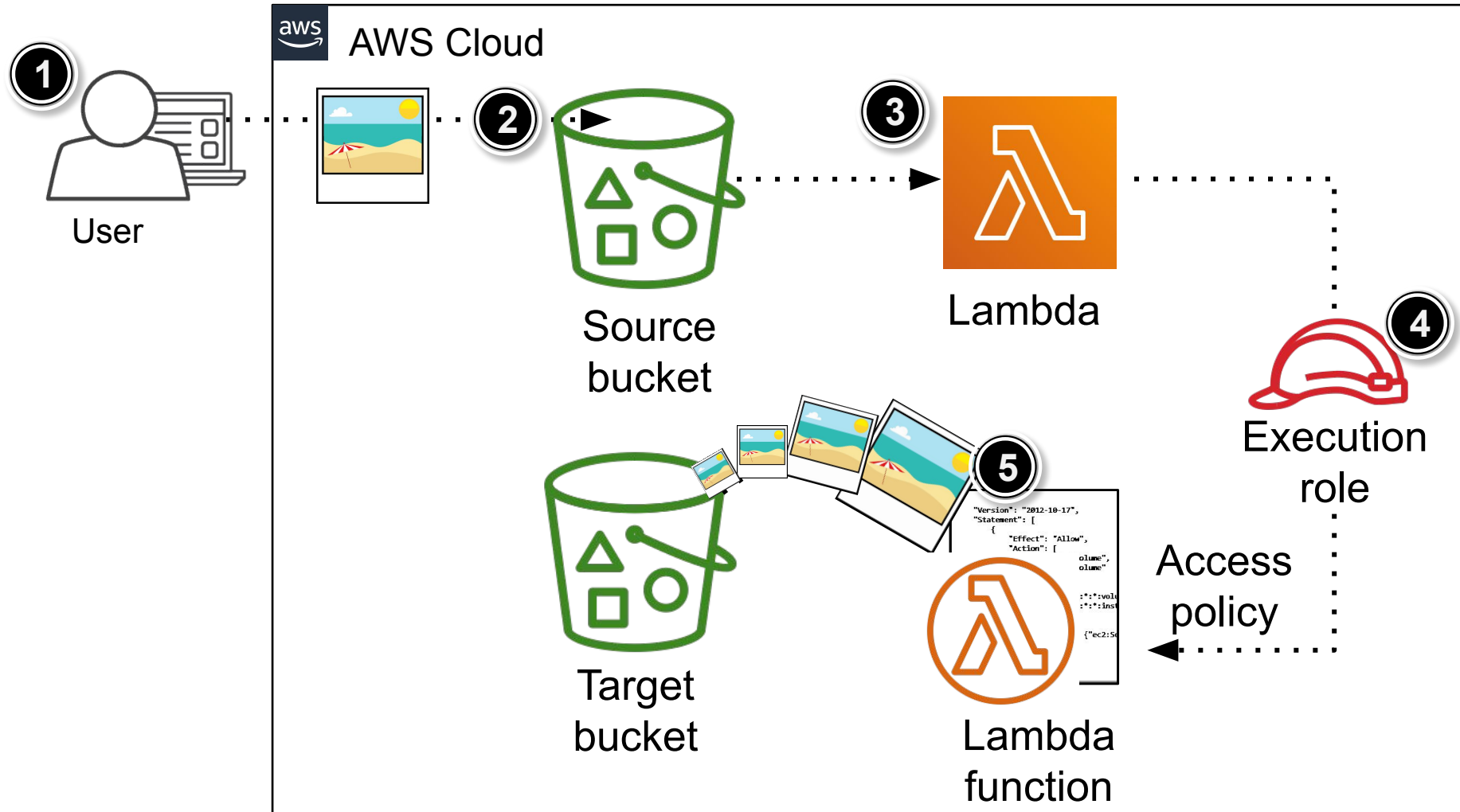
Stop instances example



Start instances example



Event-based Lambda function example: Create thumbnail images



AWS Lambda quotas

Soft limits per Region:

- Concurrent executions = 1,000
- Function and layer storage = 75 GB

Hard limits for individual functions:

- Maximum function memory allocation = 10,240 MB
- Function timeout = 15 minutes
- Deployment package size = 250 MB unzipped, including layers
- Container image code package size = 10 GB

Additional limits also exist. Details are in the AWS Lambda quotas documentation at <https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>.

Section 5 key takeaways



- **Serverless computing** enables you to build and run applications and services without provisioning or managing servers.
- **AWS Lambda is a serverless compute service** that provides built-in fault tolerance and automatic scaling.
- An **event source** is an AWS service or developer-created application that triggers a Lambda function to run.
- The maximum memory allocation for a single Lambda function is 10,240 MB.
- The maximum run time for a Lambda function is 15 minutes.

Activity: Create an AWS Lambda Stopinator Function

To complete this activity:

- Go to the hands-on lab environment and launch the AWS Lambda activity.
- Follow the instructions that are provided in the hands-on lab environment.

Activity debrief: key takeaways



Section 6: Introduction to AWS Elastic Beanstalk

Module 6: Compute

AWS Elastic Beanstalk

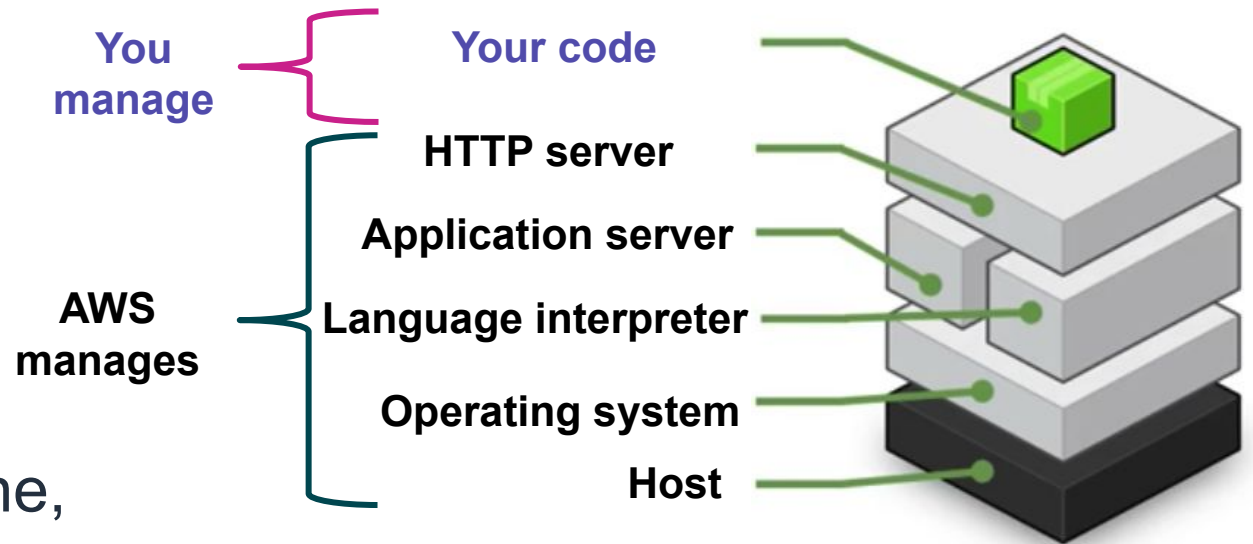


AWS Elastic Beanstalk

- An easy way to get **web applications** up and running
- A **managed service** that automatically handles –
 - Infrastructure provisioning and configuration
 - Deployment
 - Load balancing
 - Automatic scaling
 - Health monitoring
 - Analysis and debugging
 - Logging
- No additional charge for Elastic Beanstalk
 - Pay only for the underlying resources that are used

AWS Elastic Beanstalk deployments

- It supports web applications written for common platforms
 - **Java**, **.NET**, **PHP**, **Node.js**, **Python**, **Ruby**, **Go**, and **Docker**
- You upload your code
 - Elastic Beanstalk automatically handles the deployment
 - Deploys on servers such as Apache, NGINX, Passenger, Puma, and Microsoft Internet Information Services (IIS)



Benefits of Elastic Beanstalk



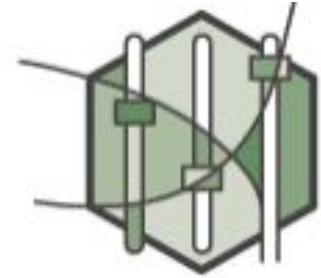
Fast and simple to
start using



Developer
productivity



Difficult to
outgrow



Complete resource
control

Activity: AWS Elastic Beanstalk

To complete this activity:

- Go to the hands-on lab environment and launch the AWS Elastic Beanstalk activity.
- Follow the instructions that are provided in the hands-on lab environment.

Activity debrief: Key takeaways



Section 6 key takeaways



- **AWS Elastic Beanstalk** enhances developer productivity.
 - Simplifies the process of deploying your application.
 - Reduces management complexity.
- Elastic Beanstalk supports **Java**, **.NET**, **PHP**, **Node.js**, **Python**, **Ruby**, **Go**, and **Docker**
- There is no charge for Elastic Beanstalk. Pay only for the AWS resources that you use.

Module wrap-up

Module 6: Compute



Module summary

In summary, in this module, you learned how to:

- Provide an overview of different AWS compute services in the cloud
- Demonstrate why to use Amazon Elastic Compute Cloud (Amazon EC2)
- Identify the functionality in the Amazon EC2 console
- Perform basic functions in Amazon EC2 to build a virtual computing environment
- Identify Amazon EC2 cost optimization elements
- Demonstrate when to use AWS Elastic Beanstalk
- Demonstrate when to use AWS Lambda
- Identify how to run containerized applications in a cluster of managed servers

Complete the knowledge check



Sample exam question

Which AWS service helps developers quickly deploy resources which can make use of different programming languages, such as .NET and Java?

Choice	Response
A	AWS CloudFormation
B	AWS SQS
C	AWS Elastic Beanstalk
D	Amazon Elastic Compute Cloud (Amazon EC2)

Sample exam question answer

Which AWS service helps developers quickly deploy resources which can make use of different programming languages, such as .NET and Java?

The correct answer is C.

The keywords in the question are developers quickly deploy resources and different programming languages.

Additional resources

- Amazon EC2 Documentation: <https://docs.aws.amazon.com/ec2/>
- Amazon EC2 Pricing: <https://aws.amazon.com/ec2/pricing/>
- Amazon ECS Workshop: <https://ecsworkshop.com/>
- Running Containers on AWS: <https://containersonaws.com/>
- Amazon EKS Workshop: <https://www.eksworkshop.com/>
- AWS Lambda Documentation: <https://docs.aws.amazon.com/lambda/>
- AWS Elastic Beanstalk Documentation:
<https://docs.aws.amazon.com/elastic-beanstalk/>
- Cost Optimization Playbook:
https://d1.awsstatic.com/pricing/AWS_CO_Playbook_Final.pdf

END